

ADR-007-test-framework-architecture

ADR-007: Test Framework Architecture

Status

Accepted

Date

2026-01-08

Context

HelixCode is a complex distributed system that requires comprehensive testing at multiple levels:

- 1. **Unit Tests:** Test individual functions and components
- 2. **Integration Tests:** Test component interactions with real dependencies
- 3. **E2E Tests:** Test complete workflows from user perspective
- 4. **Challenge Tests:** Validate AI code generation capabilities
- 5. **Performance Tests:** Ensure system meets latency/throughput requirements
- 6. **Security Tests:** Verify security controls and OWASP compliance

The testing framework must: - Support testing against multiple LLM providers - Validate generated code actually compiles and runs - Handle distributed worker scenarios - Provide comprehensive coverage reporting - Enable parallel test execution - Support CI/CD integration

Decision

We implemented a multi-tier testing architecture with specialized frameworks for different testing needs:

Test Organization

```
tests/
├── automation/      # Hardware and automation tests
├── e2e/             # End-to-end tests
│   ├── challenges/  # AI code generation challenge tests
│   ├── core/        # Core E2E functionality tests
│   ├── mocks/       # Mock services (LLM, Slack)
│   ├── orchestrator/ # Test orchestration
│   ├── phase2/      # Integration phase tests
│   ├── phase3/      # Performance phase tests
│   └── test-bank/    # Reusable test definitions
├── integration/     # Integration tests
├── memory/          # Memory system tests
├── performance/     # Performance benchmarks
└── qa/              # Quality assurance tests
```

└─ regression/	# Regression tests
└─ security/	# Security tests (OWASP)
└─ testinfra/	# Test infrastructure utilities
└─ unit/	# Unit tests

Unit Testing

Standard Go testing with testify assertions:

```
func TestExample(t *testing.T) {
    result := doSomething()
    assert.Equal(t, expected, result)
    assert.NoError(t, err)
}
```

Location: Alongside source files (manager_test.go next to manager.go)

Challenge Testing Framework

The most unique aspect: an AI code generation validation framework.

```
type ChallengeSpec struct {
    ID            string          `json:"id"`
    Name          string          `json:"name"`
    Description    string          `json:"description"`
    Type          ChallengeType   `json:"type"`
    Prompt        string          `json:"prompt"`
    Language      string          `json:"language"`
    Timeout       Duration        `json:"timeout"`

    Requirements struct {
        Files          []string `json:"files"`
        HasTests       bool     `json:"has_tests"`
        CompilationCheck bool     `json:"compilation_check"`
        TestsPass      bool     `json:"tests_pass"`
        RunCheck       bool     `json:"run_check"`
    } `json:"requirements"`
}
```

Challenge Types: - CLI applications - Web applications - REST APIs - Microservices - CRUD applications - Libraries - Full-stack applications - Bots - Games - Machine learning projects

Execution Modes: - Single instance - 2-worker distributed - 5-worker distributed - 10-worker distributed

Provider Testing: Challenges can be executed against any configured LLM provider: - Local: Ollama, Llama.cpp, vLLM, LocalAI - Cloud: OpenAI, Anthropic, Gemini, Mistral, Qwen, xAI, Groq, Azure, Bedrock, VertexAI, OpenRouter

Challenge Execution

```
type ChallengeExecution struct {
    ID                string
    ChallengeID       string
    Interface          ChallengeInterface // cli, tui, rest,
                        websocket, desktop
    Distribution       ChallengeDistribution
    Provider           LLMProviderType
    Model              string
    StartTime          time.Time
    Duration           time.Duration
    Status             ExecutionStatus
    ResultDir          string
    ValidationResults []ValidationResult
    Metrics            ExecutionMetrics
}

type ExecutionMetrics struct {
    FilesGenerated      int
    LinesOfCode         int
    Requests            int
    TokensUsed          int
    CompilationTime     time.Duration
    TestExecutionTime   time.Duration
    PlaceholdersFound   int
    EmptyFunctions      int
    CoveragePercent     float64
}
```

Validation Pipeline

1. **File Validation:** Check expected files exist
2. **Compilation Validation:** Verify code compiles
3. **Test Validation:** Run tests, check they pass
4. **Runtime Validation:** Execute the application
5. **Functional Validation:** Verify functionality
6. **Use Case Validation:** Test specific use cases

Mock Services

Mock LLM Provider: - OpenAI-compatible API - Configurable responses - Request/response logging - Latency simulation

```
// handlers/completions.go
func HandleCompletions(w http.ResponseWriter, r *http.Request) {
    // Parse request
    // Generate mock response
}
```

```
    // Optionally add latency
}
```

Mock Notification Services: - Slack webhook mock - Message capture and verification

Test Orchestration

```
type TestOrchestrator struct {
    Executor    *Executor
    Scheduler    *Scheduler
    Reporter     *Reporter
    Validator    *Validator
}
```

Features: - Parallel test execution - Test scheduling and prioritization - Multiple report formats (JSON, JUnit) - Result aggregation

Configuration

```
type ChallengeConfig struct {
    HelixCodeHost string
    HelixCodePort int
    HelixCodeAuth string

    ResultsBaseDir string
    LogsBaseDir     string

    MaxConcurrent    int
    DefaultTimeout   time.Duration
    RetryCount        int

    ValidateCompilation bool
    ValidateTests       bool
    ValidateRun         bool
    StrictValidation    bool

    VerboseLogging      bool
    SaveAllRequests      bool
    SaveAllResponses     bool
    SaveIntermediateStates bool
}
```

Test Commands

```
# Unit tests
make test
go test -v ./internal/auth

# Single test
```

```
go test -v ./internal/auth -run TestSpecific

# Coverage
make test-coverage
go test -cover ./...

# Benchmarks
make test-benchmark

# All tests
./run_all_tests.sh

# Challenge tests
cd tests/e2e/challenges
go run cmd/runner/main.go -list
go run cmd/runner/main.go -challenge notes-project-001
```

Test Infrastructure

Shared utilities in `tests/testinfra/`: - Test database setup - Mock service initialization - Test data generators - Assertion helpers

Consequences

Positive

1. **Comprehensive Coverage:** Tests at all levels
2. **AI Validation:** Unique ability to validate generated code
3. **Provider Testing:** Test against multiple LLM providers
4. **Distributed Testing:** Validate multi-worker scenarios
5. **CI/CD Integration:** JUnit/JSON output for pipelines
6. **Parallelization:** Efficient test execution
7. **Regression Prevention:** Comprehensive test suite

Negative

1. **Complexity:** Multiple test frameworks to maintain
2. **Duration:** Challenge tests are time-consuming
3. **Dependencies:** Some tests require external services
4. **Flakiness Risk:** AI-based tests may have variance
5. **Resource Intensive:** Challenge tests use significant compute

Neutral

1. **Learning Curve:** Team needs to understand multiple frameworks
2. **Maintenance:** Test code requires maintenance like production code

Alternatives Considered

Alternative 1: Single Framework (Go Testing Only)

Description: Use only standard Go testing for everything.

Pros: - Simplicity - No additional dependencies - Standard approach - Good tooling

Cons: - Limited for E2E testing - No challenge validation - No distributed testing - Manual orchestration

Why Rejected: The unique requirements of AI code generation validation require specialized frameworks.

Alternative 2: BDD Framework (Ginkgo/Gomega)

Description: Use BDD-style testing throughout.

Pros: - Readable specifications - Nested contexts - Better organization - Parallel execution

Cons: - Learning curve - Additional dependency - Different from Go conventions - Verbose for simple tests

Why Rejected: Standard Go testing with testify provides sufficient structure. BDD adds complexity without proportional benefit.

Alternative 3: External Test Platform (Cypress, Playwright)

Description: Use browser-based E2E testing tools.

Pros: - Mature ecosystems - Visual testing - Good debugging - Record and playback

Cons: - Focused on web UI - Not applicable to CLI/API - Different language (JavaScript) - Additional infrastructure

Why Rejected: HelixCode is primarily CLI/API focused. Browser-based tools don't address core testing needs.

Alternative 4: Contract Testing (Pact)

Description: Use contract testing for API verification.

Pros: - Provider/consumer contracts - Independent testing - Clear API boundaries - Good for microservices

Cons: - Additional infrastructure - Contract maintenance - Not applicable to AI generation - Overkill for current architecture

Why Rejected: Current architecture is monolithic. Contract testing better suited for microservices decomposition.

Implementation Notes

- Unit tests use testify (github.com/stretchr/testify)
- Mock interfaces in `internal/mocks/`
- Test configs in `config/test-config.yaml` and `config/minimal-config.yaml`
- Challenge definitions in `tests/e2e/challenges/challenges/`
- Results stored in `tests/e2e/challenges/test-results/`

Test Data Management

- Tests create temporary directories
- Database tests use transactions with rollback
- External service mocks replace real services
- Challenge results preserved for analysis

Continuous Integration

Test pipeline stages: 1. Unit tests (fast, always run) 2. Integration tests (medium, on PR) 3. E2E core tests (slow, on merge) 4. Challenge tests (very slow, nightly) 5. Performance tests (scheduled) 6. Security tests (scheduled)

Related Decisions

- ADR-001: LLM Provider Interface (provider testing)
- ADR-002: Distributed Worker Architecture (distributed testing)
- ADR-004: Workflow Execution Model (workflow testing)

References

- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/tests/e2e/challenges/types.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/tests/e2e/challenges/executor.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/tests/e2e/challenges/validator.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/tests/e2e/challenges/README.md`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/tests/testinfra/testinfra.go`